

Nama : Novia Fitri

NPM : 2410010284

Kelas : 5C Teknik Informatika

Eksperimen:

1. Tambahkan baris berikut di dalam method main: `Koleksi x = new Koleksi("X01", "Uji", 2026);`

Apa pesan error dari compiler dan mengapa?:

Pesan error: **COMPILATION ERROR** :

`id/ac/uniska/pbo2/p02/AplikasiPerpustakaan.java:[15,21] id.ac.uniska.pbo2.p02.Koleksi is abstract; cannot be instantiated. 1 error.`

Mengapa: Karena koleksi merupakan kelas abstract, jadi gabisa dibuat objek secara langsung pakai new. Harusnya objek dibuat dari class turunannya seperti Buku atau Majalah.

2. Pada kelas Buku, ubah nama method `hitungDenda` menjadi `hitungdenda`. Apa yang terjadi jika anotasi `@Override` tetap ada, dan apa yang terjadi jika anotasi tersebut dihapus?

Yg terjadi jika `@Override` tetap ada: akan terjadi error karena `hitungdenda()` gak lagi punya nama yang sama dengan method yang di override dari parent class `Koleksi`. Jadi, java anggap method itu bukan override.

Jika `@Override` dihapus: tidak terjadi error, tapi `hitungdenda()` Cuma jadi method baru di `Buku` dan tidak meng override `hitungDenda` yang ada pada milik `Koleksi`. Akibatnya, polymorphism untuk method itu tidak berjalan seperti seharusnya.

3. Tambahkan: `new Buku("B009", "", 2020, "Anonim");`

Apa yang terjadi saat program dijalankan?: Yang terjadi saat di run muncul pesan error `Exception in thread "main" java.lang.IllegalArgumentException: Judul tidak boleh kosong`. Itu disebabkan karena input yang dimasukan ("") judul buku diisi kosong. Sedangkan di class `Koleksi` ada aturan bahwa judul gak boleh kosong. Jadi program berhenti dan gak bisa lanjut.

4. Ubah: `private StatusKoleksi status` menjadi: `public StatusKoleksi status` Lalu ubah status `B002` langsung dari main menjadi `TERSEDIA` saat masih dipinjam. Aturan apa yang dilanggar?: Ketika status variabel diubah menjadi `public`, kita bisa langsung mengubah status `B002` dari method main menjadi **TERSEDIA**, padahal `B002` sebenarnya masih dipinjam oleh Siti. Akibatnya, Budi juga bisa meminjam `B002` karena sistem menganggap buku tersebut sudah ada. Hal ini melanggar konsep enkapsulasi. Data status yang seharusnya tidak bisa diubah sembarangan dari luar class justru dapat diakses dan diubah secara langsung.